

Endian Checker: A CPU & Memory Walkthrough

How `is_little_endian()` works at the byte level

Step 1 : The Program Starts

The CPU begins executing `main()`. When it reaches `if (is_little_endian())`, it performs a `CALL is_little_endian` and jumps into the function.

Step 2 : Creating the Variable

```
uint32_t value = 0x01020304;
```

The CPU allocates 4 bytes on the stack (assume address `0x1000`, since `uint32_t` = 4 bytes). The value `0x01020304` breaks into bytes 01, 02, 03, 04 but the byte **order** in memory depends on the CPU architecture.

Little Endian (least significant byte first):

Address	Data
0x1000	04
0x1001	03
0x1002	02
0x1003	01

Big Endian (most significant byte first):

Address	Data
0x1000	01
0x1001	02
0x1002	03
0x1003	04

Step 3 : Taking the Address

```
unsigned char *ptr = (unsigned char *)&value;
```

&value; the CPU doesn't care about the value, only the address: `&value; == 0x1000`.

(unsigned char *) changes nothing in memory; it just tells the compiler to treat that address as pointing to a single byte.

So `ptr = 0x1000` note it holds the *address*, not the value.

Step 4 : Reading One Byte and Comparing

```
*ptr          // Load Byte from address 0x1000
return (*ptr == 0x04);
```

`*ptr` means "go to the address inside `ptr` and load one byte." On Little Endian this reads `04 → 04 == 04 → returns 1 (true)`. On Big Endian it reads `01 → 01 == 04 → returns 0 (false)`.

Why unsigned char?

This is the key trick in the code. If `ptr` were a `uint32_t *`, dereferencing it would read all 4 bytes at once (`01020304`), destroying the ordering information. `unsigned char *` forces the CPU to read exactly **one byte** at a time the only way to observe how those bytes are actually arranged in memory.

How the CPU Really Sees Memory

The CPU has no concept of `uint32_t`, `int`, or `char`. It only sees a flat sequence of addressed bytes. Types are purely compiler-level instructions that say: (1) how many bytes to read/write, and (2) how to interpret them.

Why This Matters for Packet Analysis

This is exactly the concept used when parsing network packets:

```
uint8_t *packet;  
packet[0]    // *(packet + 0) -> first byte  
packet[1]    // *(packet + 1) -> second byte
```

This is why `uint8_t*` / `unsigned char*` pointers appear everywhere in network code and operating systems they let you work with memory one byte at a time, exactly the level at which the CPU and the NIC actually operate.

A handwritten signature in blue ink, consisting of stylized, overlapping loops and lines, located in the bottom right corner of the page.